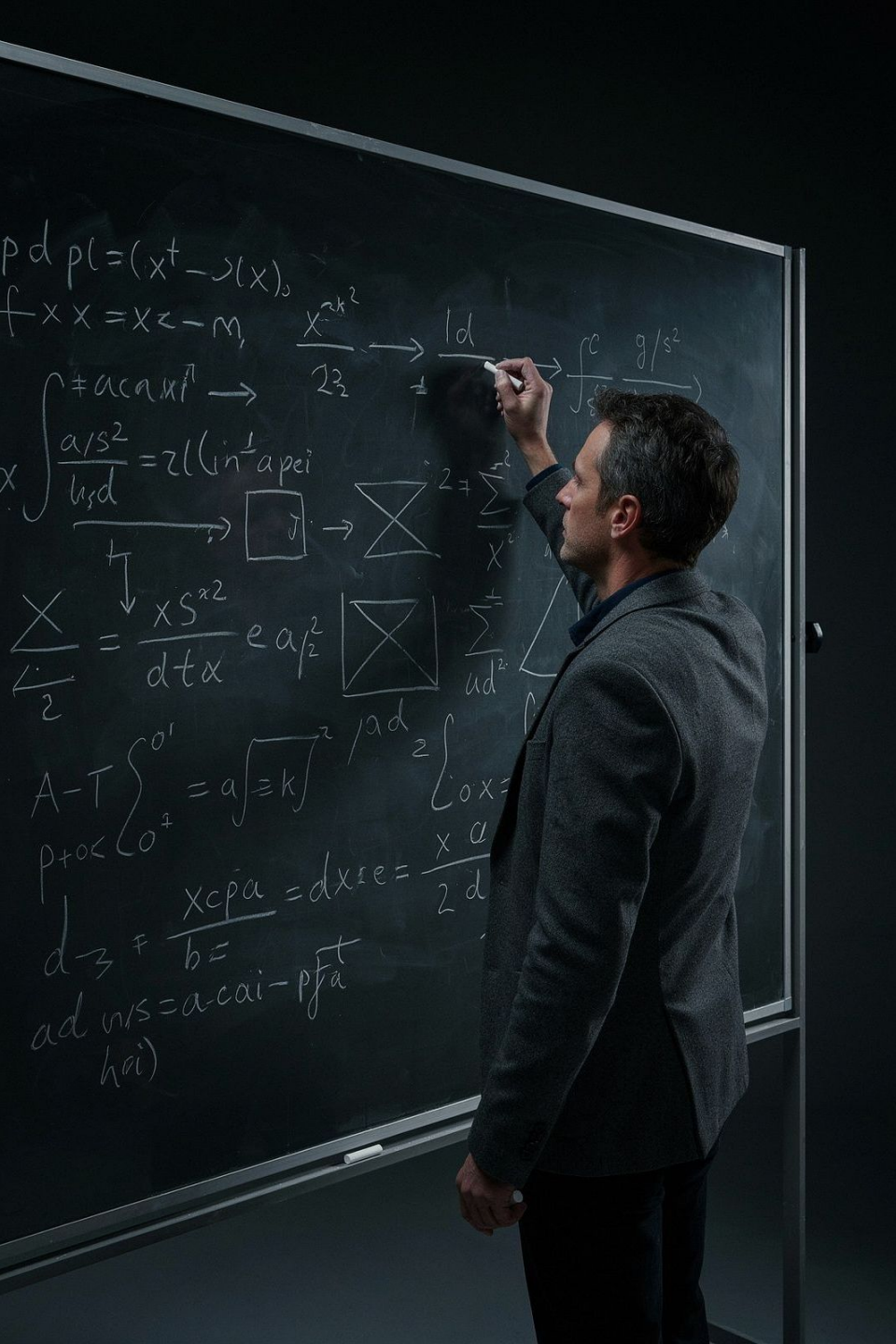




Descenso del Gradiente y Regresión desde Cero

Una exploración profunda de los fundamentos matemáticos del aprendizaje automático: desde la función de coste hasta la implementación completa de regresión lineal, paso a paso y sin atajos.

MÓDULO 5.2

NIVEL INTERMEDIO

Contenido del módulo

Un recorrido estructurado por los pilares del aprendizaje supervisado. Cada sección construye sobre la anterior, formando una base sólida antes de implementar el algoritmo completo.

01

Función de coste y gradiente

Recordatorio de los conceptos clave

03

Error cuadrático medio

La métrica de referencia en regresión

05

Tasa de aprendizaje

Control de la convergencia

02

Derivadas numéricas

Aproximación práctica sin cálculo simbólico

04

Descenso del gradiente

El algoritmo de optimización central

06

Implementación y comparación

Desde cero y con scikit-learn



¿Por qué optimizar?

Todo modelo de aprendizaje automático tiene un objetivo: **minimizar el error** entre sus predicciones y los valores reales. La optimización es el proceso de encontrar los parámetros que logran ese mínimo. El descenso del gradiente es, con diferencia, el algoritmo más utilizado para conseguirlo.

Recordatorio: Función de Coste y Gradiente

Una **función de coste** $J(\theta)$ mide cuánto se equivoca el modelo. Queremos encontrar los parámetros θ que minimizan J . El **gradiente** $\nabla J(\theta)$ es el vector de derivadas parciales de J con respecto a cada parámetro: apunta en la dirección de máximo crecimiento.

La intuición geométrica es clave: si J define una superficie en el espacio de parámetros, el gradiente indica la pendiente más pronunciada hacia arriba. Para descender, nos movemos en la dirección opuesta.

❏ El gradiente es a la optimización lo que el mapa topográfico al montañero: nos dice hacia dónde subir (y, por tanto, cómo bajar).

Propiedades del gradiente

- Es un vector con tantas componentes como parámetros tenga el modelo
- Su módulo indica la magnitud de la pendiente
- Es cero en los mínimos (y máximos) de la función
- Solo existe si J es diferenciable

Derivadas Numéricas

Cuando la expresión analítica del gradiente es compleja o desconocida, podemos **aproximarlo numéricamente**. El método más común es la *diferencia finita centrada*:

$$\frac{\partial J}{\partial \theta_i} \approx \frac{J(\theta + h \cdot e_i) - J(\theta - h \cdot e_i)}{2h}$$

Ventajas

- No requiere derivar analíticamente
- Funciona con cualquier función evaluable
- Útil para verificar gradientes analíticos (*gradient checking*)

Limitaciones

- Muy costosa computacionalmente: $O(n)$ evaluaciones por paso
- Susceptible a errores de redondeo si h es muy pequeño
- No escalable a redes neuronales grandes

Un valor típico de h es $1e-5$. En la práctica, las derivadas numéricas se usan exclusivamente para depuración, no para entrenamiento en producción.

Error Cuadrático Medio (MSE)

El **Error Cuadrático Medio** es la función de coste estándar en regresión. Mide la media de los cuadrados de las diferencias entre predicciones y valores reales:

$$J(\theta) = \text{MSE} = \frac{1}{m} \sum_{i=1}^m \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

¿Por qué cuadrados?

Penaliza errores grandes de forma desproporcionada y produce una función diferenciable en todo su dominio.

¿Por qué la media?

Normaliza el coste respecto al tamaño del conjunto de datos, haciendo comparables los valores entre distintos datasets.

¿Convexo?

Para regresión lineal, el MSE es una función convexa en θ : tiene un único mínimo global, lo que garantiza la convergencia.



La Geometría del MSE

Para regresión lineal, la superficie del MSE tiene forma de **paraboloide**: una cuenco con un único mínimo global. Esta convexidad es lo que hace que el descenso del gradiente siempre converja a la solución óptima, independientemente del punto de partida.

Descenso del Gradiente: El Algoritmo

El descenso del gradiente actualiza iterativamente los parámetros moviéndose en la dirección opuesta al gradiente. La regla de actualización es:

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j} \quad \text{para todo } j$$

1

1. Inicializar

Asignar valores iniciales a θ (normalmente ceros o valores aleatorios pequeños)

2

2. Calcular J y ∇J

Evaluar la función de coste y su gradiente con los parámetros actuales

3

3. Actualizar θ

Aplicar la regla de actualización simultáneamente para todos los parámetros

4

4. Comprobar convergencia

Repetir hasta que ΔJ sea menor que un umbral ϵ o se alcance el número máximo de iteraciones

Descenso del Gradiente: Paso a Paso

Para regresión lineal con hipótesis $\hat{y} = \theta_0 + \theta_1 x$, las actualizaciones concretas en cada iteración son:

$$\theta_0 := \theta_0 - \alpha \cdot \frac{2}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})$$

$$\theta_1 := \theta_1 - \alpha \cdot \frac{2}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) \cdot x^{(i)}$$

- ❏ Es fundamental actualizar θ_0 y θ_1 **simultáneamente**: calcular ambos gradientes con los valores actuales antes de aplicar ninguna actualización.

En forma vectorial compacta:

Tasa de Aprendizaje (α) y Convergencia

La tasa de aprendizaje α es el hiperparámetro más crítico del descenso del gradiente. Controla el tamaño de cada paso en el espacio de parámetros.

α demasiado pequeño

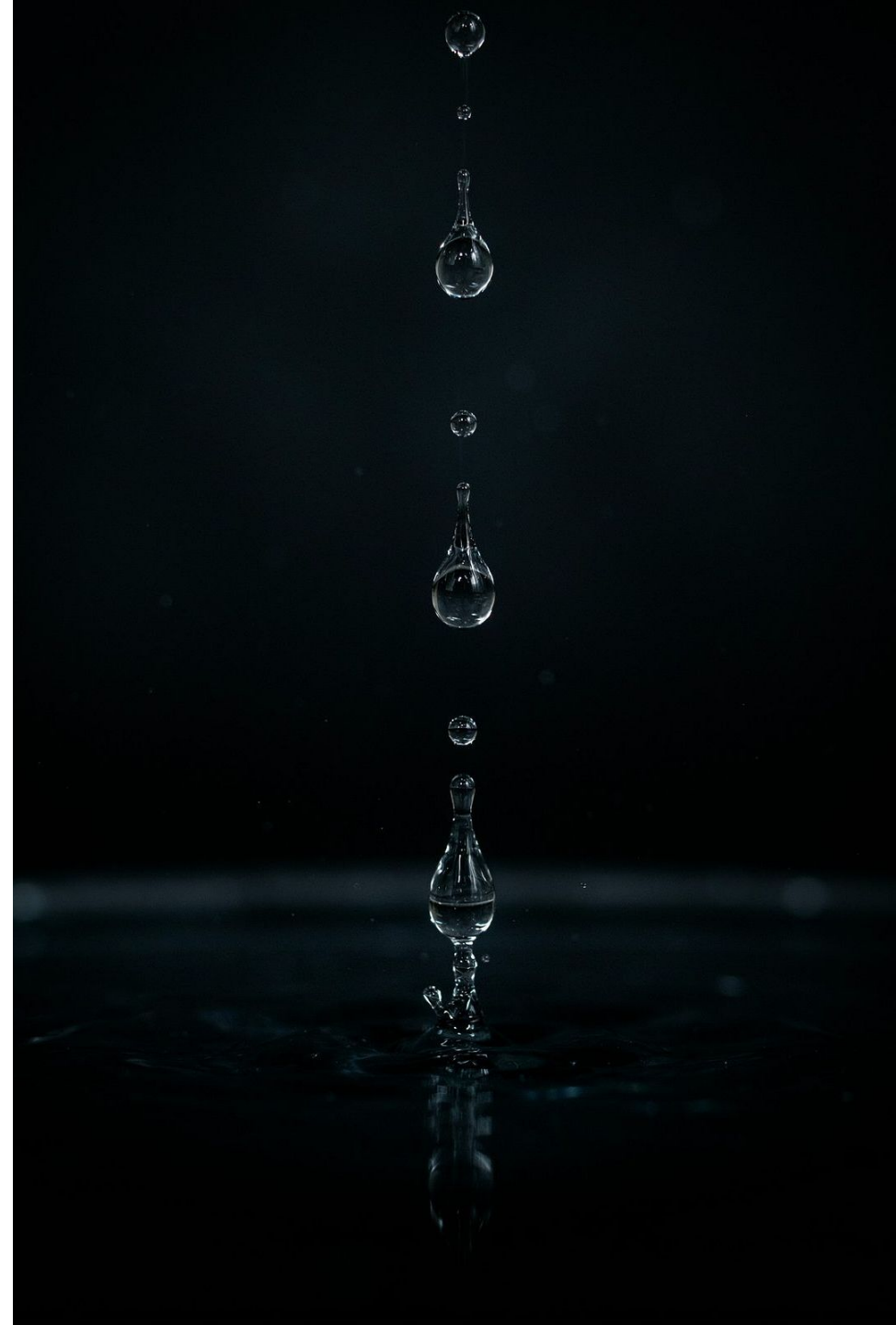
Convergencia muy lenta. El modelo aprende, pero requiere miles de iteraciones para llegar al mínimo.

α óptimo

Convergencia estable y eficiente. El coste decrece suavemente en cada iteración hasta estabilizarse.

α demasiado grande

El algoritmo oscila o diverge. El coste puede aumentar en lugar de disminuir, haciendo el entrenamiento inestable.



Estrategias para Elegir α

Búsqueda en rejilla (*Grid Search*)

Probar varios valores en escala logarítmica: 0.001, 0.003, 0.01, 0.03, 0.1, 0.3. Representar la curva de coste vs. iteraciones para cada uno y seleccionar el que converge más rápido sin oscilar.

Planificadores de tasa (*Learning Rate Schedulers*)

Reducir α progresivamente durante el entrenamiento. Estrategias comunes: decaimiento exponencial, reducción por meseta (*ReduceLROnPlateau*).

Regla empírica de diagnóstico

- Si J aumenta $\rightarrow \alpha$ demasiado grande
- Si J decrece muy lentamente $\rightarrow \alpha$ demasiado pequeño
- Si J oscila $\rightarrow \alpha$ grande o datos sin normalizar
- Si J decrece y se estabiliza $\rightarrow$ convergencia correcta ✓

 Normalizar las características con media cero y varianza unitaria acelera drásticamente la convergencia y reduce la sensibilidad a α .

Variantes del Descenso del Gradiente

Según cuántos ejemplos se usan para calcular el gradiente en cada paso, existen tres variantes con diferentes compromisos entre precisión y velocidad.



Batch GD

Usa **todo el conjunto de datos** en cada iteración. Gradiente exacto, convergencia suave, pero muy lento con datasets grandes.



Stochastic GD (SGD)

Usa **un solo ejemplo** aleatorio por iteración. Muy rápido, pero con alta varianza: la curva de coste oscila notablemente.



Mini-batch GD

Usa un **subconjunto (batch)** de 32–256 ejemplos. Equilibra velocidad y estabilidad. Es el estándar en deep learning.

Regresión Lineal desde Cero

Implementar regresión lineal en Python puro (con NumPy) consolida la comprensión del algoritmo antes de usar bibliotecas de alto nivel. La estructura básica de la clase:

```
import numpy as np

class LinearRegressionGD:
    def __init__(self, alpha=0.01, n_iter=1000):
        self.alpha = alpha
        self.n_iter = n_iter
        self.theta = None
        self.cost_history = []

    def fit(self, X, y):
        m, n = X.shape
        # Añadir columna de unos (término independiente)
        X_b = np.c_[np.ones((m, 1)), X]
        self.theta = np.zeros(n + 1)

        for _ in range(self.n_iter):
            y_hat = X_b @ self.theta
            error = y_hat - y
            gradient = (2 / m) * X_b.T @ error
            self.theta -= self.alpha * gradient
            self.cost_history.append((error ** 2).mean())

    def predict(self, X):
        X_b = np.c_[np.ones((X.shape[0], 1)), X]
        return X_b @ self.theta
```

Entrenamiento y Diagnóstico Visual

Una vez implementado el modelo, es esencial representar la curva de aprendizaje para verificar que el algoritmo converge correctamente:

```
import matplotlib.pyplot as plt

# Generar datos sintéticos
np.random.seed(42)
X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X.ravel() + np.random.randn(100)

# Entrenar el modelo
model = LinearRegressionGD(alpha=0.1, n_iter=500)
model.fit(X, y)

# Visualizar convergencia
plt.plot(model.cost_history)
plt.xlabel("Iteración")
plt.ylabel("MSE")
plt.title("Curva de aprendizaje")
plt.show()

print(f" $\theta_0 = \{model.theta[0]:.4f\}$ ,  $\theta_1 = \{model.theta[1]:.4f\}$ ")
```

✔ Con $\alpha=0.1$ y 500 iteraciones, deberías obtener $\theta_0 \approx 4.0$ y $\theta_1 \approx 3.0$, muy próximos a los valores reales del generador de datos.

Comparación con scikit-learn

Nuestra implementación manual debe producir resultados equivalentes a los de `LinearRegression` de scikit-learn (que resuelve el problema analíticamente mediante la ecuación normal). Comparemos:

```
from sklearn.linear_model import LinearRegression, SGDRegressor
from sklearn.preprocessing import StandardScaler

# scikit-learn: solución analítica (ecuación normal)
sk_model = LinearRegression()
sk_model.fit(X, y)
print(f"sklearn →  $\theta_0$ ={sk_model.intercept_:.4f},  $\theta_1$ ={sk_model.coef_[0]:.4f}")

# scikit-learn: SGD (equivalente a nuestro GD)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
sgd_model = SGDRegressor(max_iter=1000, eta0=0.1, random_state=42)
sgd_model.fit(X_scaled, y)
print(f"SGD →  $\theta_0$ ={sgd_model.intercept_[0]:.4f}")
```

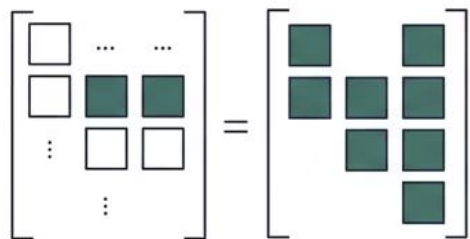
Ecuación normal (sklearn)

Solución exacta en un paso: ¹ Eficiente para datasets pequeños (<10.000 muestras). Coste $O(n^3)$ por la inversión matricial.

Descenso del gradiente

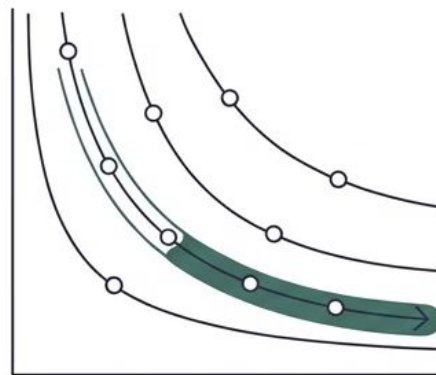
Solución iterativa: escala bien a millones de ejemplos y centenares de características. Imprescindible en deep learning donde la ecuación normal es inviable.

¿Cuándo Usar Cada Método?



ECUACIÓN NORMAL

Solución exacta.
Rápida para datos pequeños ($n < 100,000$). No escala bien a big data.



BATCH GD

Iterativo y estable.
Escala moderadamente.
Requiere ajustar la de aprendizaje (α).



MINI-BATCH SGD

El más rápido.
Escala a big data.
Estándar en deep learning. Más hiperparámetros.



Errores Comunes y Cómo Evitarlos

La implementación desde cero revela errores conceptuales que las bibliotecas de alto nivel ocultan. Conocerlos es fundamental para depurar modelos en producción.

Los 6 Errores Más Frecuentes

1. No normalizar las características

Si las variables tienen escalas muy distintas (ej. edad vs. salario), el gradiente tiene forma elongada y la convergencia es lentísima.

Solución: aplicar `StandardScaler` antes de entrenar.

2. Tasa de aprendizaje inadecuada

Un α fijo puede ser óptimo al inicio pero subóptimo al acercarse al mínimo. **Solución:** usar un scheduler o buscar α con validación cruzada.

3. Actualización no simultánea de parámetros

Actualizar θ_0 antes de usarlo en el cálculo del gradiente de θ_1 introduce un error sutil pero grave. **Solución:** guardar el gradiente completo antes de actualizar.

4. Olvidar el término independiente (bias)

No añadir la columna de unos a X hace que el modelo deba pasar por el origen. **Solución:** concatenar siempre `np.ones` o usar `fit_intercept=True`.

5. Criterio de parada incorrecto

Detenerse por número fijo de iteraciones puede ser demasiado pronto o demasiado tarde. **Solución:** combinar umbral en ΔJ con número máximo de iteraciones.

6. Confundir MSE con RMSE

El RMSE (raíz del MSE) tiene las mismas unidades que y y es más interpretable, pero su gradiente es más complejo. **Solución:** optimizar MSE y reportar RMSE como métrica.

Resumen: Del Coste al Modelo



Este flujo es la base de prácticamente todo el aprendizaje supervisado moderno, desde regresión lineal hasta redes neuronales profundas.



Conclusiones y Próximos Pasos

Lo que has aprendido

- La función de coste MSE y su geometría convexa
- Cómo aproximar gradientes numéricamente
- El algoritmo de descenso del gradiente paso a paso
- Implementación completa de regresión lineal en NumPy
- Comparación con scikit-learn y errores comunes

Próximos pasos

- Implementar mini-batch GD y comparar curvas de convergencia
- Añadir regularización L2 (Ridge) al modelo manual
- Explorar regresión logística con el mismo framework
- Introducción a retropropagación en redes neuronales



La mejor forma de consolidar estos conceptos es implementarlos desde cero al menos una vez. El código es tu cuaderno de matemáticas.